Zephyr Project

IT-Schulungen.com
Technology & Transformation

# Praxis 3:
# Eigene minimale Anwendung

1.10

# Vom leeren Ordner zum laufenden Prozess

## Fünf Schritte mit überprüfbarem Ergebnis

**01**

### Anlegen

Out-of-tree-Verzeichnis und src/ erstellen

**02**

### Beschreiben

CMakeLists.txt und prj.conf ergänzen

**03**

### Implementieren

Startmeldung und periodische Ausgabe programmieren

**04**

### Ausführen

Für native_sim bauen und zephyr.exe starten

**05**

### Vergleichen

Inkrementellen und Pristine Build erklären

Ergebnis: Eine eigene Out-of-tree-Anwendung läuft mit Zephyr-Kernel als Linux-Prozess.

# Arbeitsauftrag und Zielstruktur

PROJEKT

## zephyr-minimal/

außerhalb des Zephyr-Tree
selbstständig versionierbar
eigenes Build-Verzeichnis

KERNDATEN

## Drei Dateien

CMakeLists.txt
prj.conf
src/main.c

LAUFZEIT

## Sichtbares Verhalten

eindeutige Startmeldung
periodischer Zähler
definiertes Intervall
Ctrl+C zum Beenden

Vor dem Start festlegen: Welche Ausgabe beweist, dass wirklich die eigene Anwendung läuft?

# Projektstruktur anlegen

TERMINAL

```
$ mkdir -p zephyr-minimal/src
$ cd zephyr-minimal
$ touch CMakeLists.txt prj.conf src/main.c
$ find . -maxdepth 2 -type f | sort
```

**01 Projektwurzel**

CMakeLists.txt und prj.conf liegen direkt im Anwendungsverzeichnis.

**02 Quellverzeichnis**

Eigener C-Code liegt unter src/; generierte Dateien bleiben im Build-Verzeichnis.

Die Anwendung wird außerhalb des Zephyr-Quellbaums erstellt – nicht unter samples/.

# Startmeldung und Statusausgabe

src/main.c

```
#include <inttypes.h>
#include <zephyr/kernel.h>
#include <zephyr/sys/printk.h>

int main(void) {
  uint32_t count = 0U;
  printk("App gestartet\n");
  while (1) {
    printk("Tick %" PRIu32 "\n", count++);
    k_sleep(K_SECONDS(1));
  }
}
```

**01 Start**

Kennzeichnet die eigene Anwendung.

**02 Status**

Ein Zähler macht wiederholte Ausführung sichtbar.

**03 Intervall**

k_sleep() blockiert den Thread für die konfigurierte Dauer.

Die Ausgabe muss Start, Wiederholung und zeitliches Verhalten eindeutig erkennen lassen.

# Portable Ausgabe bewusst wählen

| | |
|---|---|
| **Zählertyp** | uint32_t hat eine definierte Breite und benötigt stdint.h oder passende Zephyr-Header |
| **Formatangabe** | PRIu32 aus inttypes.h ist für uint32_t portabel; %u nur bei passendem Typ |
| **Zeitangabe** | K_SECONDS(1) beschreibt die Wartezeit lesbar und ohne rohe Tick-Werte |
| **Schlaf statt Busy Loop** | k_sleep() gibt Rechenzeit frei; eine aktive Warteschleife blockiert unnötig |
| **Reviewfrage** | Passen Datentyp, Formatangabe und Wertebereich wirklich zusammen? |

Portabilität entsteht durch passende Typen, Makros und APIs – nicht durch Annahmen über den Host.

# Bauen, starten und beobachten

**VOLLSTÄNDIGER ENTWICKLUNGSZYKLUS**

Jeder Schritt liefert einen Nachweis, bevor die nächste Änderung beginnt.

| | |
|---|---|
| **`Konfigurieren`** | west build -b native_sim . erzeugt den ersten Build-Zustand |
| **`Bauen`** | Compiler und Linker erzeugen build/zephyr/zephyr.exe |
| **`Starten`** | west build -t run oder direkter Aufruf startet den Hostprozess |
| **`Beobachten`** | Startmeldung, Zähler und Intervall mit der Erwartung vergleichen |
| **`Beenden`** | Ctrl+C stoppt den Prozess kontrolliert für den nächsten Versuch |

Erst wenn die Ausgabe stimmt, das Intervall ändern und den nächsten Build beobachten.

# Inkrementell und Pristine vergleichen

TERMINAL

```
# Intervall ändern, dann:
$ west build
$ west build -t run

# Konfiguration bewusst neu erzeugen:
$ west build -p always -b native_sim .
```

**01 Inkrementell**

Nur Änderungen werden neu gebaut.

**02 Pristine**

Konfiguration und Build-System werden kontrolliert neu erzeugt.

Vergleicht Logdauer und ausgeführte Schritte – beide Builds müssen dieselbe aktuelle Anwendung liefern.

# Praxis 3 abgeschlossen

ERFOLG

## Eigene App läuft

Out-of-tree-Struktur erstellt

Startmeldung sichtbar

periodische Ausgabe aktiv

Buildvarianten erklärt

PARTNERREVIEW

## Nachweise prüfen

Dateien zuordnen

Ausgabe erklären

Intervall ändern

Build-Log vergleichen

ZUSATZAUFGABE

## Logging statt printk

CONFIG_LOG aktivieren

Modul registrieren

LOG_INF verwenden

Ausgabe erneut prüfen

Lernziel: Die Anwendung läuft als Linux-Prozess mit Zephyr-Kernel und nachvollziehbarem Build.